

SAP CODE
UI5 / Gateway
멘토링 4회차

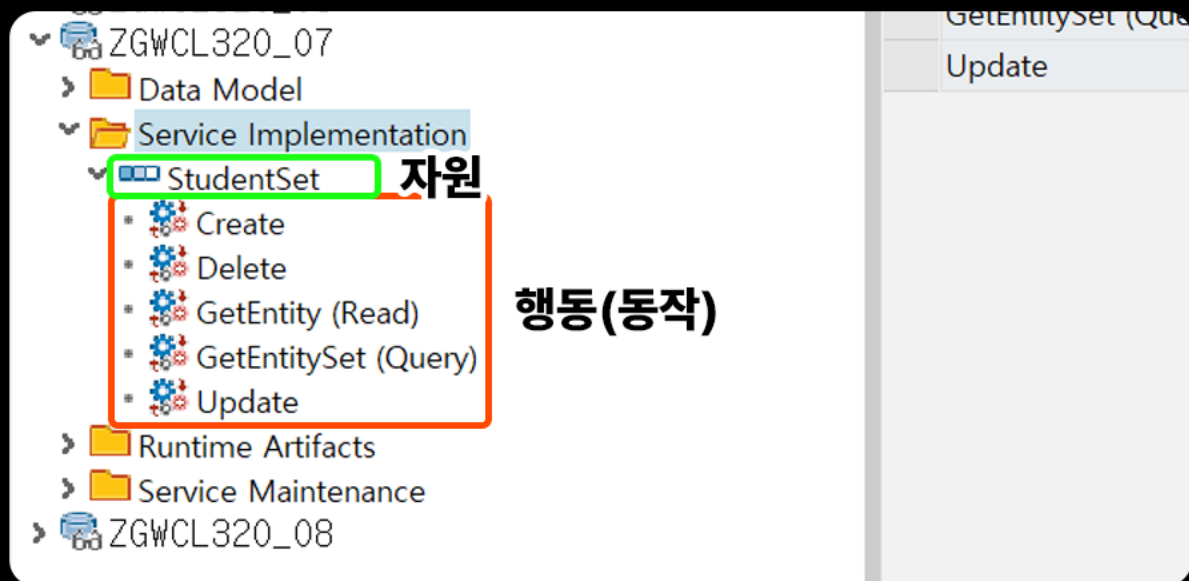
CL3 이창근

SAP

The image shows a close-up, low-angle shot of a modern building's facade. A large, blue, three-dimensional SAP logo is mounted on the upper part of the building. The logo is made of two parts: a blue 'S' and a white 'AP' on a blue background. Below the logo, there are glass windows with horizontal blinds. The sky is a clear, pale blue.

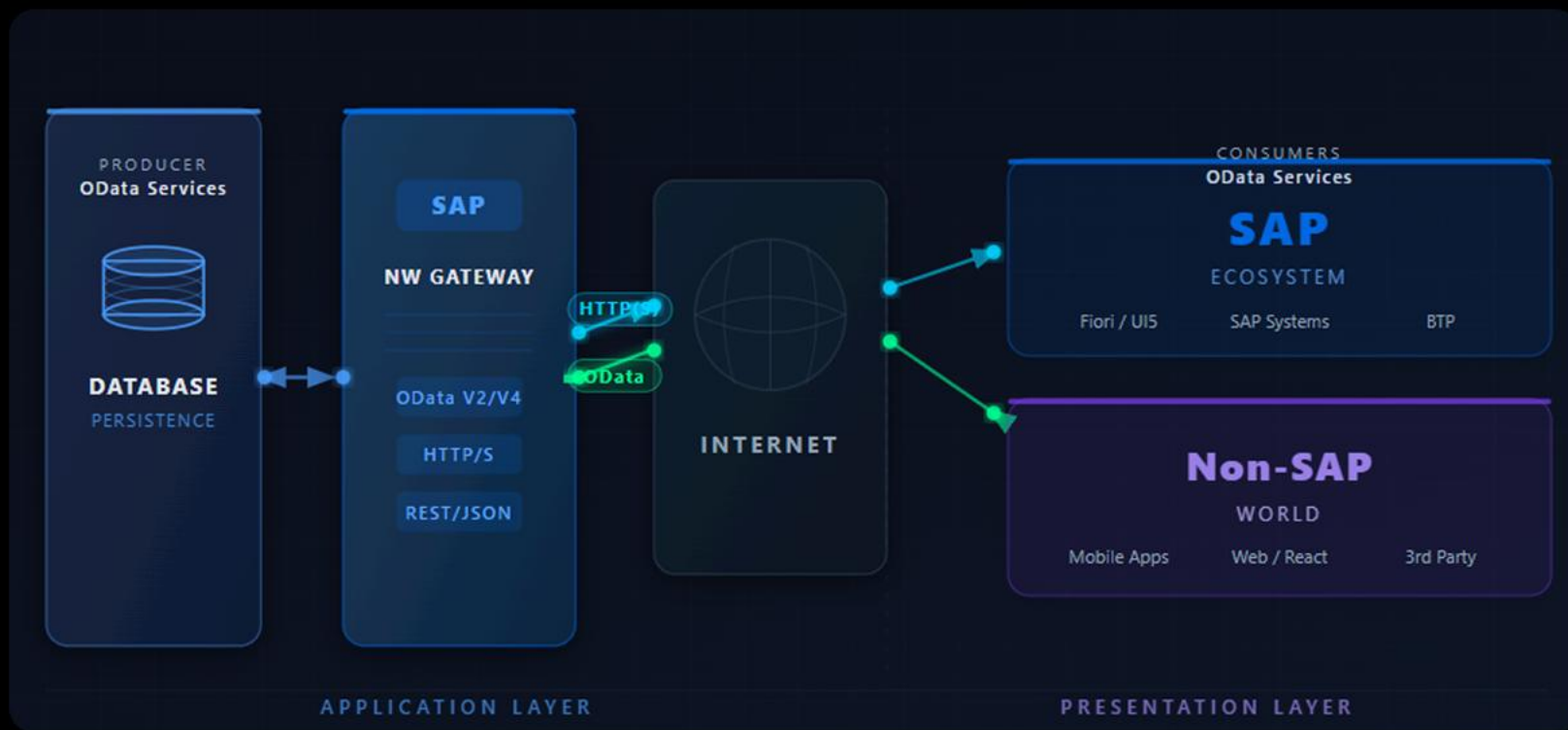
OData

- OData는 **Open Data Protocol**의 약자로, 기존 SAP의 폐쇄적인 **데이터를 외부로 노출하고 공유하도록 도와주는 프로토콜**
- OData를 사용하면 다양한 웹 브라우저, 모바일 디바이스, 기업용 소프트웨어 등과 데이터를 주고 받을 수 있음
 - **RESTful API**를 구축하기 위한 것이 목적이며, 기존의 RFC, BAPI, IDoc 중심의 내부 통신 구조에서 진보한 것
 - RESTful : Web의 장점을 최대한 활용하여, **URL로 자원을 명시하고, HTTP 메소드로 행동을 표현**하는 API를 만드는 것
 - API : 서로 다른 소프트웨어 애플리케이션이 서로 통신하고 기능을 공유할 수 있도록 하는 **연결 통로이자 규칙**
- OData를 사용하여 **SAP의 데이터를 JSON이나 XML 등의 널리 쓰이는 데이터 포맷으로 변환**할 수도 있음



SAP NetWeaver Gateway

- OData라는 프로토콜을 만들었는데, 그럼 SAP 내부는 어떻게 접근해야 하지?
 - 그것을 가능하게 해주는 것이 Gateway이다.
- 즉 Gateway는 **SAP와 외부끼리 서로 데이터를 주고받을 수 있는 일종의 창문**이라고 생각하면 이해가 편함
 - **외부에서 HTTP(s) 메시지를 보내면 SAP는 OData로 응답한다**



Gateway Service를 생성하는 두 가지 방법 : CDS View

- CDS View에서 **@OData** Annotation을 통하여 EntitySet(자원) 및 Gateway Service를 생성할 수 있다
- CDS View를 활성화하면 /IWFND/MAINT_SERVICE에서 Gateway Service를 생성할 수 있다.
 - 현재 실습환경인 OData V2에서는 GET METHOD만 가능하다.

Data Definition ZCL3_20_16_DDL [EFX, 100]

```
1 @AbapCatalog.sqlViewName: 'ZVCL3_20_21'
2 @AbapCatalog.compiler.compareFilter: true
3 @AbapCatalog.preserveKey: true
4 @AccessControl.authorizationCheck: #NOT_REQUIRED
5 @EndUserText.label: 'OData CDS View'
6 @Metadata.ignorePropagatedAnnotations: true
7
8 /* Begin of OData Annotation */
9
10 @OData.entitySet.name: 'MatSet'
11 @OData.publish: true
12
13 /* End of OData Annotation */
14
15 // Service Name : ZCL3_20_16_DDL
17 define view ZCL3_20_16_DDL as
18   select key a.matnr,
19           b.maktx,
20           a.vpsta,
21           a.mtart,
22           a.matkl
23   from mara as a inner join makt
24   on a.matnr = b.matnr;
```

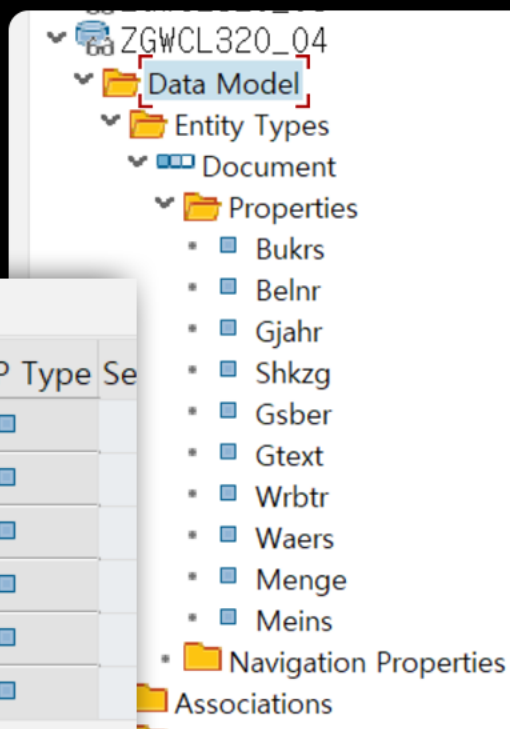
Service Catalog

Type	Technical Service Name	Vers...	Service Description	External Service Name
BEP	ZCL3_20_16_DDL_CDS	1	OData CDS View	ZCL3_20_16_DDL_CDS
BEP	ZCL3_20_17_DDL_CDS	1	Purchasing Document	ZCL3_20_17_DDL_CDS
BEP	ZCL3_20_18_DDL_CDS	1	Flight Schedule	ZCL3_20_18_DDL_CDS
BEP	ZCL3_20_19_DDL_CDS	1	CDS OData View - 2026. 02. 12 13:03	ZCL3_20_19_DDL_CDS

Gateway Service를 생성하는 두 가지 방법 : SEGW

- **T-CODE `SEGW`**에서 직접 Gateway Service 및 EntitySet도 생성할 수 있다.
- CDS View를 통하여 생성한 것과는 다르게 **CREATE / DELETE / UPDATE 등의 동작을 할 수 있다**
- EntitySet (자원) 은 **Import > DDIC Import**를 이용하여 Dictionary Object를 이용하여 생성할 수도 있고
 - **Create > Entity Type** 을 이용하여 직접 데이터 타입을 정의하여 생성할 수도 있다.

Properties			
Name	ABAP Field	ABAP Type	Se
Carrid	CARRID	☐	
Connid	CONNID	☐	
Fldate	FLDATE	☐	
Price	PRICE	☐	
Currency	CURRENCY	☐	
Planetype	PLANETYPE	☐	



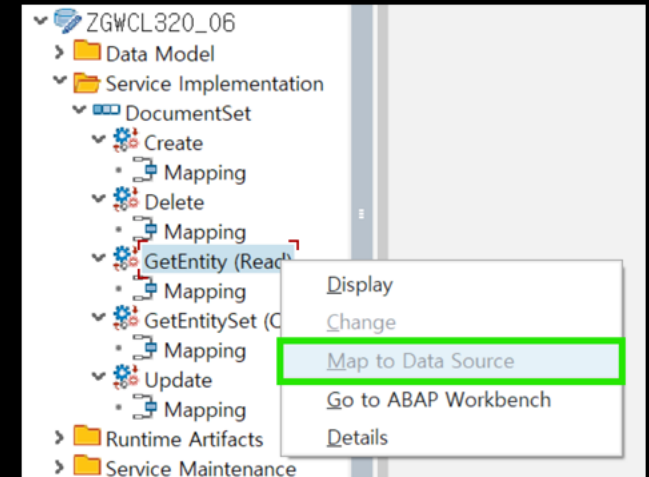
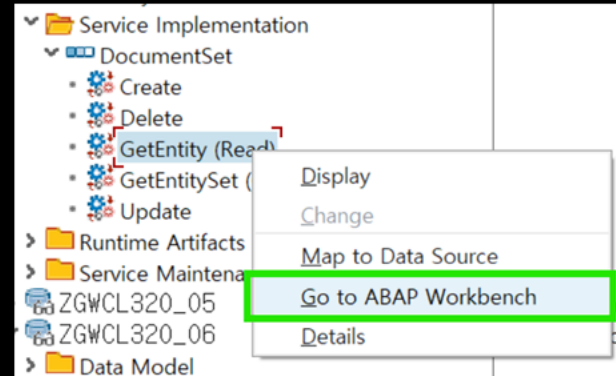
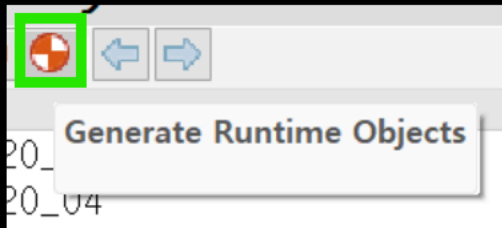
주의사항

- 시간/날짜는 `Edm.String`으로 타입을 꼭 바꿔서 적어주기
- Key Fields는 `Updatable`과 `Nullable` 체크해제하기
- DDIC Import의 경우 **DB의 필드명에 _(언더바)가 들어가 있을 경우 stdt_no → Stdtno로 변환됨**
- 데이터가 나오지 않을 경우 Name과 ABAP Field 꼭 확인해보기

SEGW에서 Gateway Service의 '동작'을 구현하는 방법

- 각 Gateway Service의 'Service Implementation'에서 CRUD를 구현할 수 있다.

- Data Model 및 EntitySet을 전부 정의한 후 **Generate Runtime Objects** 버튼을 선행으로 눌러주어야 한다.



- 각 '동작'을 우클릭하여 둘 중 하나를 선택하면 된다.

- **각 EntitySet마다 존재하는** DPC_EXT 클래스의 메소드를 재정의하여 구현하는 방법
- 'Map to Data Source'를 이용하여 각 동작에 Remote Function Call을 Mapping하여 구현하는 방법
 - **Function Module 정의 시 Remote-Enabled로 설정하면 외부에서 호출 가능한 FM이 된다.**
 - **왜 쓰나요?** → Class보단 FM이 코딩이 '비교적' 자유롭기도 하고, 이전에 만든 RFC를 재사용하기 위함
 - 기존에 잘 만들어진 RFC FM이 있는데, Gateway를 위해 이것 전부 Class로 새로 짜는건 쉽지 않음

T-CODE : /IWFND/GW_CLIENT

▪ Gateway Service를 직접 실행해볼 수 있는 클라이언트

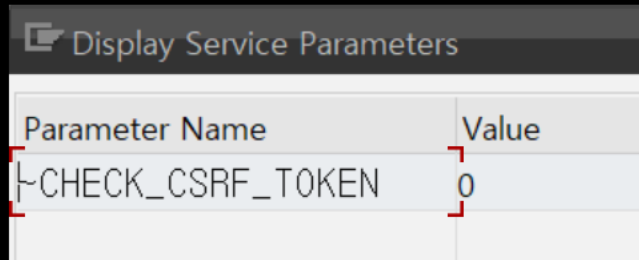
- 각 HTTP Method는 Gateway Service의 행위(Method 혹은 RFC)와 연결된다.

Service Implementation	HTTP Method	
GetEntity	GET	/sap/opu/odata/sap/서비스이름/엔티티셋이름(PK전부적기) <code>/sap/opu/odata/sap/ZGWCL320_03_SRV/AccountSet(Bukrs='1000',Belnr='1000000001',Gjahr='2010',Buzei='001')</code>
GetEntitySet	GET	/sap/opu/odata/sap/서비스이름/엔티티셋이름 <code>/sap/opu/odata/sap/ZGWCL320_03_SRV/AccountSet</code>
CREATE	POST	서비스이름/엔티티셋이름(PK전부적기) 쓰고 F8 누른 후 Use as Request 좌측 하단 HTTP Request 부분에서 내용 완성한 후 F8 다시 누르기
DELETE	DELETE	서비스이름/엔티티셋이름(PK전부적기) 쓰고 F8 PK를 전부 적어야 해당 행을 유니크하게 식별하고 삭제할 수 있음
UPDATE	PATCH/MERGE	서비스이름/엔티티셋이름(PK전부적기) 쓰고 F8 누른 후 Use as Request 좌측 하단 HTTP Request 부분에서 PK를 제외한 내용 을 수정한 후 F8 다시 누르기

V2 : MERGE (공식 HTTP Method 아님)
V4 : PATCH (진짜 있는 HTTP Method임)

CSRF Token : 애 왜 있는거야?

- 우리를 귀찮게 하는 CSRF 토큰(~CHECK_CSRF_TOKEN)은 대체 왜 있는걸까?
 - 없으면 Deploy도 안된다고 그러지, Gateway Client에서 오류를 띄우기도 하는 등...
- 사실 이건 보안때문에 사용하는 것
 - URL에 모든 자원을 담는 OData(RESTful API) 특성 상 DB의 내용이 모두 드러나있음
 - https를 사용한다면 URL이 암호화되어 안전해지지만 http를 사용할 경우 보호할 수단이 없음
 - 만일 외부에서 해커가 URL를 탈취하게 된다면 DB의 안전을 보장할 수 없어지게 된다
 - 그래서 최소한의 안전장치를 마련하기 위해 걸어두는 보호라고 생각하면 된다



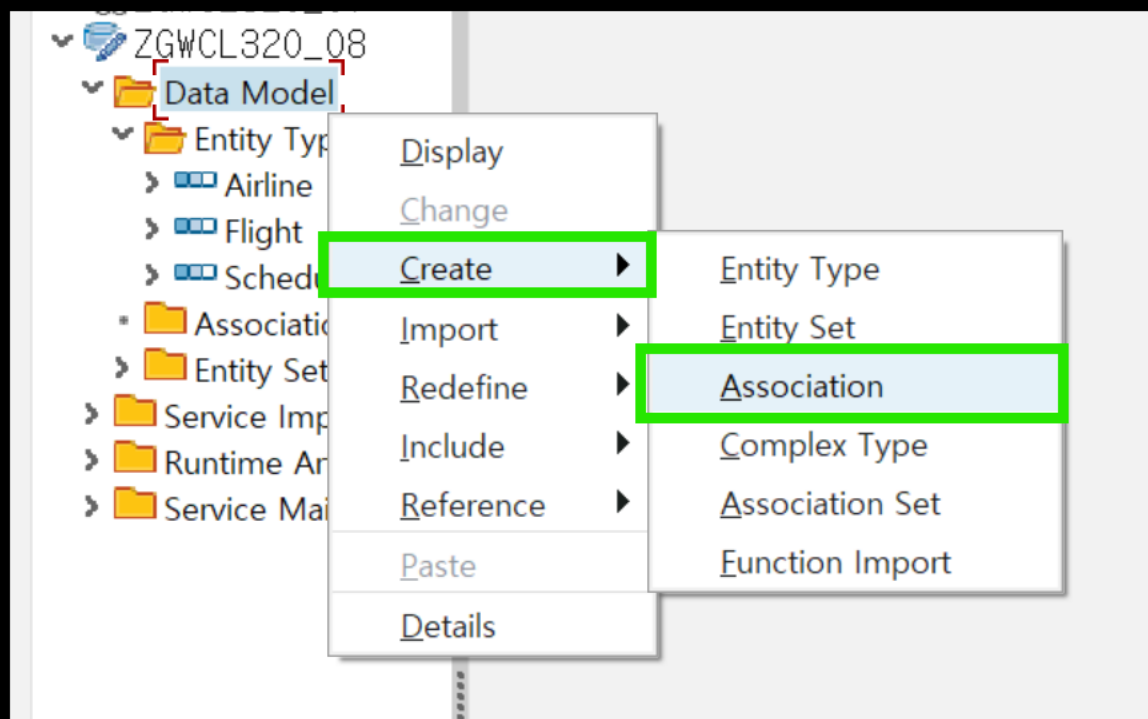
Parameter Name	Value
~CHECK_CSRF_TOKEN	0

0 추가 / 제거 스탠다드 Function

23	▶				
24	▶		CALL FUNCTION	'CONVERSION_EXIT_ALPHA_INPUT'	
25	▶		EXPORTING		
26	▶		input	=	
27	▶	[-]	IMPORTING		
28	▶	[-]	OUTPUT	=	빈자리를 0으로 추가하는 함수
29	▶				
30	▶				
31	▶		CALL FUNCTION	'CONVERSION_EXIT_ALPHA_OUTPUT'	
32	▶		EXPORTING		
33	▶		input	=	
34	▶	[-]	IMPORTING		
35	▶	[-]	OUTPUT	=	0을 제거하는 함수
36	▶				

어제 사실 이런 것도 가능하긴 했어요

- EntitySet을 3개 만들고, 각 EntitySet의 GetEntitySet 메소드도 3개를 재정의했던 2월 25일 실습
 - 사실 **SEGW에서 각 EntitySet의 Association을 생성하여 관계를 정의했다면 메소드를 1개로 줄일 수 있다**
 - AirlineSet은 상위 엔터티셋, ScheduleSet과 FlightSet은 하위 엔터티셋으로 정의가 되기 때문
 - 다만 이렇게 하면 구현이 너무 복잡해지고, RAP라는 완벽한 대체제가 있기 때문에 가르쳐주지 않으신 듯



Fiori에서 CRUD 하기

- 먼저 **this.getView().getModel();**을 이용하여 oModel 변수를 선언
 - 메인 서비스가 아닌 별도로 선언한 oModel이라면 getModel 괄호 안에 매개변수로 oModel의 이름을 대입
 - let oModel2 = **this.getView().getModel("products");**
- oData 변수도 선언 후, HTTP 메소드에 따라 직접 JSON 형태로 만들거나, getObject() 등을 사용하여 oData를 만들기
- oModel의 CRUD 메소드를 사용하면 끝
 - read(sPath, mParameters)
 - create(sPath, oData, mParameters)
 - update(sPath, oData, mParameters)
 - delete(sPath, mParameters)
 - sPath는 **URI Request**를 작성하는 부분, oData는 앞서 만든 oData 변수
 - mParameters는 일반적으로 success function (성공했을 때 일어나는 일들)과 error function(실패했을 때)를 작성

GetEntity (GET Method)

```
onDisplay() {  
    MessageToast.show("Displayed");  
    let oTable = this.getView().byId("air"); // 테이블 선언  
    let aIndex = oTable.getSelectedIndices(); // 테이블에서 선택한 행의 인덱스를 가져옴  
    let oData = oTable.getContextByIndex(aIndex[0]).getObject(); // 해당 인덱스의 OData  
    let oModel = this.getView().getModel();  
  
    oModel.read(`/AirlineSet('${oData.Carrid}')`, {  
        success(oReturn) {  
            this.getView().byId("Carrid").setValue(oReturn.Carrid);  
            this.getView().byId("Carrname").setValue(oReturn.Carrname);  
            this.getView().byId("Currcode").setValue(oReturn.Currcode);  
            this.getView().byId("Url").setValue(oReturn.Url);  
        }.bind(this),  
  
        error() {  
            MessageToast.show("Read Error");  
        },  
    });  
},
```

GetEntitySet (GET Method)

```
onSpfli(oEvent) {  
    var oData = oEvent.getSource().getBindingContext().getObject();  
    let oBinding = this.getView().byId("spfli").getBinding("rows");  
    let aFilter = [];  
  
    aFilter.push(new Filter("Carrid", FilterOperator.EQ, oData.Carrid));  
    oBinding.filter(aFilter);  
},
```

Create (POST Method)

```
onCreate() {  
  let oModel = this.getView().getModel();  
  
  let oData = {  
    Carrid: this.getView().byId("Carrid").getValue(),  
    Carrname: this.getView().byId("Carrname").getValue(),  
    Currcode: this.getView().byId("Currcode").getValue(),  
    Url: this.getView().byId("Url").getValue(),  
  };  
  
  oModel.create(`/AirlineSet`, oData, {  
    success() {  
      oModel.refresh();  
      MessageToast.show("Create Success");  
    },  
    error() {  
      MessageToast.show("Create Error");  
    },  
  });  
},
```

Update (MERGE/PATCH Method)

```
onUpdate() {  
    let oModel = this.getView().getModel();  
  
    let oData = {  
        Carrid: this.getView().byId("Carrid").getValue(),  
        Carrname: this.getView().byId("Carrname").getValue(),  
        Currcode: this.getView().byId("Currcode").getValue(),  
        Url: this.getView().byId("Url").getValue(),  
    };  
  
    oModel.update(`/AirlineSet('${oData.Carrid}')`, oData, {  
        success() {  
            oModel.refresh();  
            MessageToast.show("Update Success");  
        },  
        error() {  
            MessageToast.show("Update Error");  
        },  
    });  
},
```

Delete (DELETE Method)

```
onDelete() {  
  let oModel = this.getView().getModel();  
  let vCarrid = this.getView().byId("Carrid").getValue();  
  
  if (!vCarrid) {  
    MessageToast.show("Please Enter Airline Code");  
  } else {  
    oModel.remove(`/AirlineSet('${vCarrid}')`, {  
      success() {  
        oModel.refresh();  
        MessageToast.show("Delete Success");  
      },  
      error() {  
        MessageToast.show("Delete Error");  
      },  
    });  
  }  
},
```

실습문제 1

- 개인이 가지고 있는 학생관리 테이블을 복사해서 새로 생성한다.
- STDT_NO 말고도 MAJOR도 KEY 필드로 만든다.
 - 복수전공인 학생이 있을 수도 있으니까.
 - 복수전공인 학생을 대략 3~4명 정도 넣어보자
- 해당 테이블을 DDIC Import하여 새로운 GW 서비스를 생성한다.
 - DPC_EXT의 메소드는 재정의하여 만든다.
 - Gateway Client에서 꼭 테스트해보고 가기
- 이전 실습에서는 스킵하고 넘어갔었던 테이블 검색 기능(GetEntitySet)을 추가한다.
 - OverflowToolbar에 Input 태그 2개와 press 이벤트를 발생시킬 버튼을 하나 추가하여 구현한다.

430

431 Transparent Table ZTCL3_00_01 Active

432 Short Description Student management

433 Attributes Delivery and Maintenance Fields Input Help/Check Currency/Quantity Fields

434

435

436

437

438

439

440

441

442

443

444

445 위의 테이블을 이용하여 UI5 Grid Table 및 유지보수 하는 기능을 구현한다.

446 Gateway 부터 새로 생성한다.

Field	K...	In...	Data element	Data Type	Len...	Deci...	Coordinate	Short Description
MANDT	☒	☒	MANDT	CLNT	3	0	0	Client
STDT_NO	☒	☒	ZEC3_STDT_NO07	CHAR	10	0	0	학번
MAJOR	☐	☐	ZEC3_MAJOR_07	CHAR	2	0	0	전공
STDT_NAME	☐	☐	ZEC3_STDT_NAME_07	CHAR	10	0	0	이름
ADDR	☐	☐	ZEC3_ADDR_07	CHAR	50	0	0	주소
EMAIL	☐	☐	ZEC3_EMAIL_00	CHAR	30	0	0	Email
GENDER	☐	☐	ZEC3_GENDER_07_01	NUMC	2	0	0	Gender

실습문제 2

- bkpfl를 이용하여 bseg를 조회하는 Fiori 프로젝트를 만듭니다.
 - 하나의 페이지에서 두 개의 Grid Table이 같이 출력되도록 합니다.
 - bkpfl 테이블의 셀을 클릭하면 cellClick 이벤트가 일어나게 하여 bkpfl의 데이터를 이용해 bseg를 조회하도록 하세요
- 한 서비스에 EntitySet을 2개 만들어도 되고, 두 서비스를 만들어서 각각 EntitySet을 만들어도 됩니다.
 - 후자가 더 어려운 난이도일거임 (심화).
 - sap/ui/model/odata/v2/ODataModel를 사용해서 두번째 oModel를 만들어야 두 개의 서비스를 사용 가능
- bkpfl entitySet은 buks(K), belnr(K), gjahr(K), blart, bldat (날짜) 총 5개의 필드
- bseg entitySet은 buks(K), belnr(K), gjahr(K), buzei(K), shkzg, pswbt(Amount), pswsl(Currency)
 - **EntitySet 만들 때 bldat 타입에 주의, pswbt와 pswsl은 UI5에서 단위 처리하여 하나의 열에 같이 출력하기**

실습문제 3

- Grid Table을 1개 만들고 테이블 하단에 Panel과 그 안에 Text/Input 필드를 각각 7개 추가하세요.
- Grid Table은 SBOOK의 전체 내용을 출력하게 하고, 셀을 클릭하면 SBOOK의 CUSTOMID를 이용하여 SCUSTOM 테이블의 Unique한 행 하나를 가져와서 Input 필드에 값을 세팅합니다. (SetValue)
 - Input 필드는 '읽기 전용'으로 설정합니다.
- EntitySet을 만들 때 SBOOK의 FLDATE와 ORDER_DATE의 데이터 Type에 주의합니다.